

ChronoVault

Timelock Encryption System — Security Architecture & Documentation

Field	Value
Document Version	2.2
Application Version	ChronoVault v2.2
Date	19.07.2026
Classification	Technical Documentation
Supersedes	v2.1 (22.12.2025)

Contents

Contents	2
1. Executive Summary.....	3
Key Security Properties	3
2. Data Privacy & Network Communication.....	3
What Stays Local (Never Transmitted).....	3
Network Communication	3
Example brand Request	4
3. Security Architecture	4
Layered Encryption Model	4
Client-Side Security Model.....	4
4. Cryptographic Standards.....	4
5. brand Integration.....	5
How the Timelock Works	5
Network Configuration (quicknet)	5
6. Security Features	6
Password Verification (v2.2)	6
Timelock Feedback.....	6
Additional Measures	6
7. Risks & Limitations.....	6
Critical Risks.....	6
Design Limitations	7
8. Best Practices.....	7
For Users.....	7
For Deployment	8
9. Technical Specifications	8
Vault File Format (v2.2).....	8
Supported Browsers	8
10. Version History	9

1. Executive Summary

ChronoVault is a client-side timelock encryption application. It combines password-based encryption with a cryptographic timelock powered by the drand distributed randomness beacon operated by the League of Entropy. A user selects data (text or a file such as a PDF), a password, and a future unlock time; ChronoVault produces a portable .vault file that cannot be decrypted by anyone — including its creator — before the chosen time.

Since v2.0, the timelock is a genuine cryptographic mechanism based on identity-based encryption (tlock/IBE), not a client-side clock check. Decryption before the target time is computationally infeasible without the drand beacon signature for the target round, which does not exist until that round is published.

Key Security Properties

- Real cryptographic timelock: the data key is IBE-encrypted to a future drand round (tlock scheme); no bypass exists in the client code.
- AES-256-GCM authenticated encryption for confidentiality and integrity of the payload.
- PBKDF2-SHA256 key derivation (100,000 iterations for the encryption key; 200,000 iterations for the password verifier).
- Zero server-side storage — all cryptographic operations run in the browser via the Web Cryptography API.
- Layered encryption: an inner password layer and an outer timelock layer with independently derived keys.
- v2.2: the fast SHA-256 password hash has been removed from the vault format, eliminating an offline brute-force vector (see Section 10).

2. Data Privacy & Network Communication

✓ USER DATA NEVER LEAVES THE COMPUTER

All encryption and decryption operations are performed entirely within the user's web browser using the Web Cryptography API and the tlock-js library. No plaintext, password, or key material is ever transmitted to any server.

What Stays Local (Never Transmitted)

- Your secret text or file content
- Your password
- All derived and generated encryption keys
- The encrypted .vault file

Network Communication

ChronoVault performs exactly two kinds of network requests, neither of which contains user data:

- **drand API (api.drand.sh):** read-only GET requests that fetch public chain information, the current round number, and (at unlock time) the public beacon signature for the target round. Responses are public randomness data available to anyone; requests could be cached or proxied without security impact.

- **Library CDN (esm.sh, fallback cdn.skypack.dev):** at startup the application dynamically loads the open-source tlock-js and drand-client libraries. This transfers no user data, but it does mean the loaded code is trusted at runtime; see Section 7 and the deployment recommendations in Section 8.

Example drand Request

```
GET https://api.drand.sh/52db9ba7.../public/latest
```

```
Response: { "round": 351804210, "randomness": "abc123...",  
            "signature": "8d61..." }
```

3. Security Architecture

Layered Encryption Model

ChronoVault v2.x encrypts data in three steps. The password layer sits inside the timelock layer, so even the correct password reveals nothing before the unlock time.

Layer	Protection	Mechanism	Purpose
1 (Inner)	Password	AES-256-GCM; key from PBKDF2-SHA256, 100,000 iterations, 256-bit random salt	User authentication
2 (Outer)	Timelock	AES-256-GCM with a fresh random 256-bit symmetric key	Encrypts the password-layer ciphertext
3 (Key wrap)	Timelock (IBE)	tlock identity-based encryption of the Layer-2 key to the target drand round	Makes the Layer-2 key unrecoverable before unlock time

At decryption time the flow runs in reverse: the password is verified first (Section 6), the drand beacon for the target round is fetched and cryptographically verified, tlock recovers the symmetric key, Layer 2 is decrypted, and finally Layer 1 is decrypted with the password-derived key. If the target round has not yet been published, step 3 fails cryptographically — there is nothing in the vault file or the client code that can shortcut it.

Client-Side Security Model

- **No server trust required:** plaintext and keys never leave the client.
- **Hardware-backed randomness:** salts, IVs, and the symmetric key come from `crypto.getRandomValues` (OS/hardware CSPRNG).
- **Authenticated primitives:** AES-GCM detects any tampering with ciphertexts; drand beacons are BLS-verified against the pinned chain public key.

4. Cryptographic Standards

Component	Algorithm	Standard / Basis	Parameters
Symmetric encryption	AES-256-GCM	NIST SP 800-38D	256-bit key, 96-bit IV
Key derivation (data key)	PBKDF2-HMAC-SHA256	NIST SP 800-132	100,000 iterations, 256-bit salt

Component	Algorithm	Standard / Basis	Parameters
Password verifier	PBKDF2 + HMAC-SHA256	NIST SP 800-132 / FIPS 198-1	200,000 iterations, dedicated 256-bit salt
Timelock encryption	tlock (IBE on BLS12-381)	tlock paper (ePrint 2023/189), RFC 9380 hashing	Encrypt-to-round, scheme bls-unchained-g1-rfc9380
Beacon signatures	BLS threshold signatures	drand / League of Entropy	Verified against pinned chain public key
Random generation	CSPRNG	NIST SP 800-90A	OS/hardware entropy via Web Crypto

5. drand Integration

drand is a decentralized randomness beacon operated by the League of Entropy, whose members include Cloudflare, EPFL, Kudelski Security, and Protocol Labs. A threshold of independent operators must cooperate to produce each beacon; no single member can predict or forge one.

How the Timelock Works

1. drand quicknet publishes a BLS threshold signature (“beacon”) for consecutive round numbers every 3 seconds.
2. Round r is published at $\text{genesisTime} + (r - 1) \times \text{period}$. Future round numbers are therefore known in advance, but their signatures are not.
3. ChronoVault computes the round corresponding to the requested unlock time and IBE-encrypts the symmetric key to that round number as the identity, using the network’s public key.
4. The beacon signature for the target round acts as the IBE decryption key. Until drand publishes it, decryption is computationally infeasible for everyone.
5. At unlock time, any copy of the beacon (from any drand relay) allows decryption; there is no dependency on ChronoVault infrastructure.

Network Configuration (quicknet)

Property	Value	Security Implication
Network	quicknet	Production network, unchained scheme required by tlock
Chain hash	52db9ba7...c84e971	Pins the chain; beacons verified against it
Round period	3 seconds	Unlock time resolution of one round
Genesis time	1692803367 (Unix)	Deterministic round $\leftrightarrow$ time mapping
Scheme	bls-unchained-g1-rfc9380	Unchained beacons enable encrypt-to-future-round
Verification	BLS signature check, enabled	Beacons are not trusted from HTTP alone

Note on timing precision: unlock times are quantized to 3-second rounds. Depending on rounding, the vault may become decryptable up to one round period (~3 seconds) around the requested wall-

clock time. For any realistic use of ChronoVault this is negligible, but sub-second unlock precision is not a design goal.

6. Security Features

Password Verification (v2.2)

ChronoVault verifies the password before consulting the timelock, so a wrong password is rejected immediately instead of after the unlock time. The verifier is deliberately slow to resist offline guessing:

- **Verifier:** HMAC-SHA256 over a fixed domain-separation constant, keyed by PBKDF2-SHA256 (200,000 iterations) of the password.
- **Dedicated salt:** the verifier uses its own 256-bit random salt (verifierSalt), independent from the encryption salt, isolating it from the data key derivation.
- **Removed in v2.2:** earlier versions additionally stored a fast reference hash (SHA-256 of password + salt) in the vault file. This allowed offline password brute-forcing at raw SHA-256 speed, bypassing the slow verifier entirely. The field has been removed; the slow verifier is now the only password-verification data in the vault.

Timelock Feedback

Condition	User Feedback
Wrong password, before unlock time	Password incorrect. Content cannot be revealed before [date/time].
Correct password, before unlock time	Password correct. Content cannot be revealed before [date/time].
Wrong password, after unlock time	Decryption failed — password incorrect.
Correct password, after unlock time	Success — data decrypted and returned.

Additional Measures

- Fresh 256-bit salts and 96-bit IVs generated for every vault; no key or IV reuse across layers or vaults.
- Authenticated encryption (GCM) on both layers detects any modification of the ciphertext.
- Legacy v1 vaults (which stored a bypassable timelock secret) are still readable but are clearly flagged as insecure; new vaults are always v2.2.

7. Risks & Limitations

⚠ IMPORTANT: Users should understand these limitations before relying on ChronoVault for critical data.

Critical Risks

Risk	Severity	Description	Mitigation
Password loss	CRITICAL	A lost password means permanent data loss. There is no recovery mechanism.	Use a password manager
Vault file loss	CRITICAL	A deleted or corrupted .vault file cannot be reconstructed.	Keep multiple backups
drand network failure	HIGH	If quicknet permanently stopped, timelocked vaults could become unopenable. Temporary API outages only delay unlocking.	Decentralized operator set; beacons are mirrored by multiple relays
Runtime-loaded libraries	MEDIUM	tlock-js and drand-client are fetched from public CDNs at page load without integrity pinning. A compromised CDN could serve malicious code at encryption time.	Self-host / vendor the libraries; see Section 8
Browser environment	MEDIUM	Malicious extensions or a compromised device can read the plaintext and password while the page is open.	Use a clean, trusted browser
Weak password	MEDIUM	Anyone holding the vault file can test password guesses offline against the slow verifier (~200 ms per guess).	Use 12+ character passwords

Design Limitations

- **Cleartext metadata:** the vault file exposes its type, creation time, unlock time, and — for file vaults — the original filename. The content is protected; the metadata is not. Do not use sensitive filenames.
- **Trust in the League of Entropy:** a colluding threshold of drand operators could in principle compute future beacons early. This is the scheme's explicit trust assumption.
- **Offline password guessing:** early password feedback inherently requires storing a verifier in the vault file; the 200,000-iteration PBKDF2 makes guessing slow but not impossible for weak passwords.
- **File size:** files above ~10 MB may exhaust browser memory during in-browser encryption.
- **No forward secrecy:** once the unlock time has passed, anyone holding both the vault file and the password can decrypt, forever.

8. Best Practices

For Users

6. Use strong, unique passwords (minimum 12 characters with mixed case, numbers, and symbols) — the vault file permits offline guessing.
7. Store the password in a reputable password manager; write down that the vault exists and when it unlocks.
8. Keep multiple backups of the .vault file in different locations.
9. Verify the unlock date and time carefully before encrypting; it cannot be changed afterwards.

10. Give file vaults neutral filenames — the original filename is stored in cleartext.
11. Use a clean, trusted browser without suspicious extensions, ideally offline-capable after page load.

For Deployment

12. Serve over HTTPS only.
13. Vendor tlock-js and drand-client into the deployment instead of loading them from public CDNs at runtime, or pin them with Subresource Integrity where possible.
14. Set a strict Content Security Policy limiting script sources and connect-src to api.drand.sh.
15. Re-audit the codebase after dependency updates.

9. Technical Specifications

Vault File Format (v2.2)

The .vault file is a self-contained JSON document. Everything required for decryption after the unlock time is inside it (plus the public drand beacon):

```
{
  "version": "2.2",
  "type": "text | pdf | file",
  "created": "ISO-8601 timestamp",
  "security": { "method": "tlock-ibe", "description": "..." },
  "timelock": {
    "targetRound": 351804210,
    "unlockTime": "ISO-8601",
    "chainHash": "52db9ba7...",
    "publicKey": "83cf0f28...",
    "genesisTime": 1692803367,
    "roundPeriod": 3,
    "schemeID": "bls-unchained-g1-rfc9380",
    "timelockCiphertext": { "encoding": "utf8|base64", "data": "..." }
  },
  "crypto": {
    "algorithm": "AES-256-GCM",
    "kdf": "PBKDF2",
    "iterations": 100000,
    "salt": "base64 (32 bytes)",
    "iv": "base64 (12 bytes)",
    "dataIv": "base64 (12 bytes)",
    "verifierSalt": "base64 (32 bytes)",
    "passwordVerifier": "base64 (32 bytes)"
  },
  "data": "base64-encoded double-encrypted payload",
  "originalFilename": "only for file vaults (cleartext!)"
}
```

Compatibility: v2.2 code reads v2.0 and v2.1 vaults unchanged (any legacy passwordHash field present in old vaults is ignored). v1 vaults decrypt via a legacy path and are flagged as insecure.

Supported Browsers

Browser	Minimum Version	Notes
Chrome / Edge	63+	Full Web Crypto + dynamic ES module import
Firefox	67+	Full support

Browser	Minimum Version	Notes
Safari	11.1+	Full support

10. Version History

Version	Date	Changes
2.2	07/2026	Removed the fast SHA-256 passwordHash field from the vault format (offline brute-force vector that bypassed the slow verifier). Password verification now relies solely on the PBKDF2+HMAC verifier. Documentation overhauled.
2.1	12/2025	Added slow password verifier (PBKDF2 200k + HMAC, dedicated salt) enabling early wrong-password rejection. JSON-safe timelock ciphertext serialization.
2.0	2025	Real cryptographic timelock introduced: symmetric key IBE-encrypted to a future drand round via tlock-js (quicknet). Replaced the bypassable v1 round check.
1.0	2025	Initial release. Timelock enforced only by a client-side round comparison with the secret stored in the vault file — bypassable; deprecated.

— End of Document —